

Click-to-Source

A Visual Debugging & Parameter-Tracing Tool for Procedural Three.js / React Three Fiber Scenes

Project Proposal & Technical Plan | Prepared by Punith P | August 2026

1. One-Line Summary

A developer tool that lets you click on any rendered feature inside a Three.js / React Three Fiber scene — a mountain ridge, a tree, a weather particle system, a shader-driven color — and instantly see the source code, function, and exact parameter value responsible for producing it, so it can be understood and edited directly instead of being hunted down manually or located via an AI chat query.

2. The Problem

AI-assisted development (Claude, Cursor, Copilot, etc.) is now a normal way to build creative/procedural web projects. AI tools are very good at generating a working base — terrain, particle systems, lighting, shaders — but they rarely get every value exactly right on the first pass. The result almost always needs small, precise corrections: a lake that is too deep, a forest that is too dense, a weather system with too many particles.

The problem is not generating the fix — it's **locating** the exact place in the code responsible for the specific thing you're looking at. Today this happens in one of two inefficient ways:

- **Manual search** — scrolling and grepping through generated code you may not have written yourself, trying to match a visual detail to a variable name.
- **Asking an AI to find it** — describing the visual issue in words ("the lake on the left looks too deep"), which is imprecise, burns tokens/API calls on a purely mechanical lookup task, and can take several iterations to land on the right value.

Both approaches spend disproportionate effort on a task that is fundamentally mechanical: mapping a *rendered result* back to the *exact code and value* that produced it. That mapping doesn't require reasoning or generation — it requires precise, deterministic tracing, which is a problem suited to tooling, not AI inference.

3. Core Idea / Value Proposition

Click-to-Source acts as a **precision-targeting layer** between "I want to change something specific" and the two current options (manual search or an AI call). It removes ambiguity that language cannot easily express — "that slightly-too-dense patch of trees on the left ridge" is hard to describe precisely in a prompt, but trivial to indicate with a click.

4. Why This, Why Now

- **Timing:** AI-directed development of procedural/creative scenes is now common, but tooling for correcting AI-generated output hasn't caught up — this gap is specific to how people build in 2026, not a generic, long-solved problem.

- **Founder-market fit:** This solves a pain already experienced firsthand while building the Procedural Low-Poly World Generator (terrain, shaders, per-face color hashing, weather particles) — the target user and the builder are the same person for v1, which is the strongest possible validation signal at this stage.
- **Market size check:** Three.js sees roughly 2.7M npm downloads/week with a steady, non-hype-cycle growth curve; React Three Fiber sees roughly 900K weekly downloads and is the standard way React developers build 3D web experiences. Large enough to matter, focused enough that no dedicated tooling exists yet for this specific workflow.

5. Prior Art / Competitive Landscape

A dedicated review was run to check whether this idea, or something equivalent, already exists. The ecosystem has real, mature prior art in adjacent areas, but no publicly documented tool combines per-instance parameter provenance with automatic build-time instrumentation for procedural Three.js / React Three Fiber scenes. The closest existing tools are summarized below.

Tool	What it does	What it misses
Needle Inspector	Chrome DevTools extension; auto-detects Three.js/R3F projects, shows scene hierarchy, real-time property editing, performance monitoring.	Operates at object/scene-graph level (meshes, materials, transforms) — not the arguments passed to the generator function for a specific instance.
Three.js Inspector (legacy)	Older Chrome extension for scene hierarchy and object inspection.	Predates React Three Fiber; no React component-tree awareness; no per-instance procedural tracing.
Threejs Devtools MCP Server	~59-60 tools for inspecting/modifying scenes in real time; supports click_inspect for click-to-inspect mode; integrates with AI agents (Claude Desktop, Cursor, Copilot) via MCP.	Closest existing competitor. Emphasizes object details (geometry, materials, performance) rather than mapping an instance back to generator-function arguments and exact source file/line.
InspectableR3F	R3F devtools library for inspecting HTML textures rendered on meshes; right-click to edit underlying HTML.	Solves a different problem entirely — DOM-content-as-texture inspection, not procedural geometry or shader tracing.
React click-to-component tools (Xray, OpenSail Design Engineer)	Click-to-source for React DOM trees; walks the Fiber tree or tags JSX elements to resolve a rendered element back to its source file/line.	Built for DOM elements and JSX, not Three.js meshes inside a canvas; no per-instance argument capture.
Browser source maps	Map compiled/minified JS back to original source files and line numbers, standard in all browser DevTools.	File/line mapping only — no tracking of individual runtime arguments per function call instance.

Conclusion: the ecosystem has strong prior art in scene inspection, DOM click-to-source, and AI-assisted Three.js devtools — but the specific combination this proposal targets (per-instance parameter provenance, automatic generator-function instrumentation, and a procedural-scene-focused click-to-edit UX) has no publicly documented equivalent. The Threejs Devtools MCP server is the closest existing tool and the one most worth using hands-on before further development, both to avoid duplicating effort and

because it is the most likely candidate to add this capability first if development stalls.

6. How It Works (Technical Approach)

5.1 The Core Loop

- User clicks a rendered feature in the running scene.
- A raycast identifies the exact mesh / object under the cursor.
- The tool resolves that object back to the source: file, function, and — critically — the exact runtime value(s) used to generate that specific instance (not just the line that created it).
- A side panel shows the resolved source snippet and the live value(s), editable directly.
- Editing a value updates the running scene immediately (hot-reload), closing the loop without needing an AI call.

5.2 Source Mapping — Two Layers

A naive approach (manually attaching metadata like `file / line` to every generated object) does not scale — no developer will hand-annotate every call site, and identical code run in a loop (e.g. 500 tree instances) would all resolve to the same line, losing per-instance precision. The architecture instead separates two concerns:

Layer	Solves	Mechanism
Location	Which file / function created this kind of object	Automatic build-time instrumentation (Vite/Babel AST transform) — same technique used by React DevTools' JSX source plugin. Zero manual annotation.
Instance value	Which exact parameter value produced <i>this specific</i> object, out of many generated by the same code	The build transform wraps designated generator functions to capture resolved arguments at call time, attaching a combined {file, line, function, args} record to the returned object.

Manual metadata tagging (e.g. `mesh.userData.sourceRef`) is kept only as a fallback for cases the automatic transform cannot cleanly infer — it is the v0.1 shortcut, not the long-term mechanism.

5.3 A Note on Precision

The tool deliberately avoids promising "the exact line" as a hard guarantee. Some results come from a single line; others come from a chain — a noise function feeding a mapping function feeding a shader uniform. The tool's job is to surface "the source responsible for producing this result," which may be a single value or a short traceable chain, rather than overpromising single-line precision in every case.

7. MVP Scope (Version 0.1)

Deliberately minimal, to validate the core mechanism before investing further:

- React Three Fiber only (not raw Three.js, not other engines).
- Meshes only (no shaders / materials / lighting yet).
- Manual metadata tagging (automatic Vite/Babel instrumentation comes later, after the loop is proven).

- One demo scene, adapted from the existing Procedural World Generator, with an intentionally obvious flaw (e.g. a too-deep lake) to make the before/after story legible.
- No AI integration in v0.1 — the tool must be useful entirely on its own first.

Success criterion for v0.1:

"Within 30 seconds of clicking an object, a developer who did not write the scene can find and edit the parameter responsible for that object's appearance." If this holds, the core idea is validated. If it feels clunky, that's discovered in weeks, not months.

8. Staged Development Plan

Stage	Goal	Est. Time
0 — Scope freeze	Lock the one-sentence definition of the product; anything that doesn't serve it goes on a "later" list.	< 1 day
0.5 — Light validation	A handful of informal conversations with Three.js/R3F developers about procedural-scene debugging pain, run in parallel with building — not a gate.	Ongoing
1 — Prove the mechanism	Throwaway prototype: can a runtime mesh be reliably mapped back to meaningful source info at all, using the existing Procedural World Generator?	2–3 days
2 — Metadata approach	Manual tagging convention for v0.1; design (but defer building) the automatic Vite/Babel instrumentation for later.	~1 week
3 — Core loop (MVP)	Click → identify object → show source + value → edit → live update. No AI, no polish.	2–3 weeks
4 — Dogfooding	Use the tool on the real Procedural World Generator while fixing its existing known regressions (normals ordering, weather freeze, lake bed clamp).	1–2 weeks
5 — AI handoff	Add a "send this exact resolved context to AI" action — turns the token-saving thesis into a real, demoable feature.	~1 week
6 — Polish & story	Clean UI, README framed around the problem (not the feature list), and a before/after demo narrative.	~1 week
7 — Ship	Publish on GitHub (MIT), short demo video, narrow launch to a relevant community (r/threejs, r/webdev, relevant Discords).	Ongoing

Total realistic timeline: roughly 6–9 weeks of consistent part-time work, assuming Stage 1 doesn't reveal a fundamental blocker in the mapping mechanism.

9. Longer-Term Possibilities (Post-MVP, Not Committed)

- Automatic build-time instrumentation via a Vite/Babel plugin, replacing manual metadata tagging entirely.
- Shader/material support, not just mesh geometry.
- Extending beyond React Three Fiber to raw Three.js projects.
- Linking project version history to the click-to-source view — seeing how a specific rendered feature changed across past edits.

10. Risks & Open Questions

- **Mechanism risk:** Reliable object-to-source mapping may prove harder than expected for complex/composed procedural chains — this is exactly why Stage 1 exists, as a cheap, early test.
- **Adoption assumption:** The idea assumes developers think in terms of rendered objects when debugging. This mostly holds for the builder's own R3F/mesh-based scenes, but may not generalize to everyone (e.g. developers who think more in systems, ECS, or shader pipelines) — a question for after v0.1, not before.

- **Speed bar:** The tool's value only holds if click-to-source is faster than typing an AI prompt, every single time. If it's ever slower or flaky, users (including the builder) will default back to AI chat. This is the single most important thing to stress-test in Stage 3.

Prepared by Punith P — Computer Science & Engineering, KS Institute of Technology, Bengaluru. Document generated as a working reference for personal project planning.